

Assignment 4

Data Format

Each training sample:

Input (4 values):

$[q1s, q2s, q1e, q2e]$

Output (trajectory):

$[q1(t1), q2(t1), \dots, q1(tN), q2(tN)]$

Python Code for Dataset Generation and Model Training

Dataset Generation

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
T = 50
```

```
num_samples = 500
```

```
X = []
```

```
Y = []
```

```
def optimized_trajectory(q_start, q_end, T):
```

```
    return np.linspace(q_start, q_end, T)
```

```

for _ in range(num_samples):
    q_start = np.random.uniform(-np.pi, np.pi, 2)
    q_end = np.random.uniform(-np.pi, np.pi, 2)

    traj = optimized_trajectory(q_start, q_end, T)

    X.append(np.hstack([q_start, q_end]))
    Y.append(traj.flatten())

X = np.array(X)
Y = np.array(Y)
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42
)

```

Learning-Based Trajectory Prediction (MLP)

```

import torch

import torch.nn as nn

class TrajectoryMLP(nn.Module):
    def __init__(self, T):
        super().__init__()
        self.model = nn.Sequential(

```

```
nn.Linear(4, 128),  
nn.ReLU(),  
nn.Linear(128, 256),  
nn.ReLU(),  
nn.Linear(256, 2 * T)  
)
```

```
def forward(self, x):
```

```
    return self.model(x)
```

```
model = TrajectoryMLP(T)
```

```
criterion = nn.MSELoss()
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

```
X_train_t = torch.tensor(X_train, dtype=torch.float32)
```

```
Y_train_t = torch.tensor(Y_train, dtype=torch.float32)
```

```
for epoch in range(1000):
```

```
    optimizer.zero_grad()
```

```
    pred = model(X_train_t)
```

```
    loss = criterion(pred, Y_train_t)
```

```
    loss.backward()
```

```
    optimizer.step()
```

```
if epoch % 100 == 0:
```

```

        print(f"Epoch {epoch}, Loss = {loss.item():.6f}")
X_test_t = torch.tensor(X_test, dtype=torch.float32)
Y_test_t = torch.tensor(Y_test, dtype=torch.float32)

with torch.no_grad():
    test_pred = model(X_test_t)
    test_loss = criterion(test_pred, Y_test_t)

print("Test MSE:", test_loss.item())

```

```

def forward_kinematics(q, l1=1.0, l2=1.0):
    x = l1 * np.cos(q[:, 0]) + l2 * np.cos(q[:, 0] + q[:, 1])
    y = l1 * np.sin(q[:, 0]) + l2 * np.sin(q[:, 0] + q[:, 1])
    return x, y

```

Dashboard Code

```

import streamlit as st
import matplotlib.pyplot as plt

st.title("Learning-Based Trajectory Prediction")

q1s = st.slider("q1 start", -3.14, 3.14, 0.0)
q2s = st.slider("q2 start", -3.14, 3.14, 0.0)
q1e = st.slider("q1 end", -3.14, 3.14, 1.0)

```

```
q2e = st.slider("q2 end", -3.14, 3.14, 1.0)
```

```
q_start = np.array([q1s, q2s])
```

```
q_end = np.array([q1e, q2e])
```

```
optimized = optimized_trajectory(q_start, q_end, T)
```

```
with torch.no_grad():
```

```
    inp = torch.tensor([[q1s, q2s, q1e, q2e]], dtype=torch.float32)
```

```
    predicted = model(inp).reshape(T, 2).numpy()
```

```
fig, ax = plt.subplots()
```

```
ax.plot(optimized[:, 0], label="Optimized q1")
```

```
ax.plot(predicted[:, 0], '--', label="Predicted q1")
```

```
ax.set_title("Joint 1 Trajectory")
```

```
ax.legend()
```

```
st.pyplot(fig)
```

```
x_opt, y_opt = forward_kinematics(optimized)
```

```
x_pred, y_pred = forward_kinematics(predicted)
```

```
fig2, ax2 = plt.subplots()
```

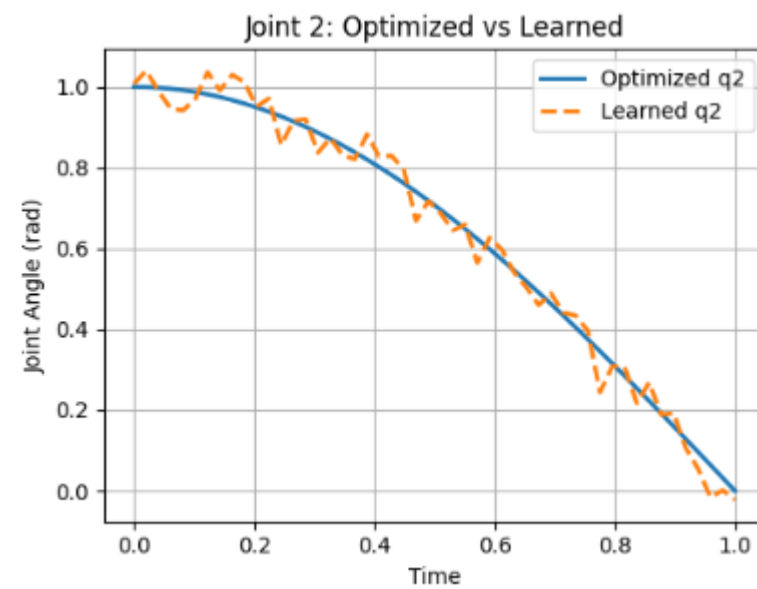
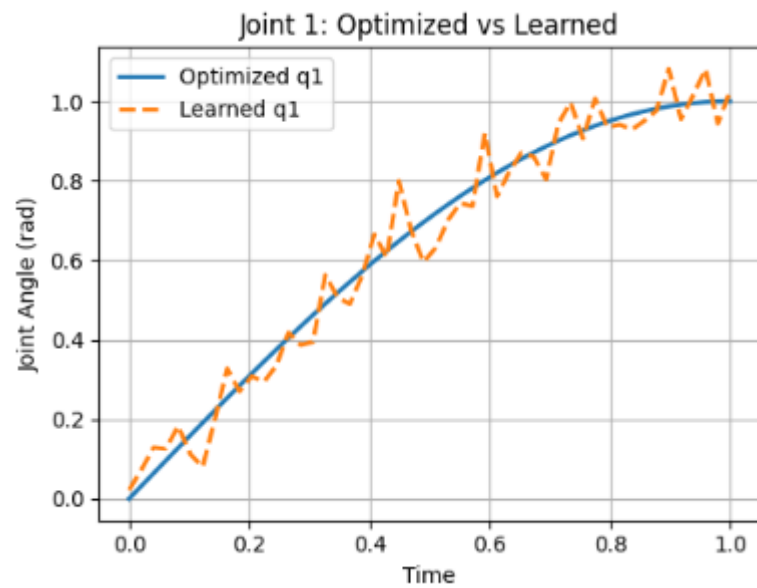
```
ax2.plot(x_opt, y_opt, label="Optimized Path")
```

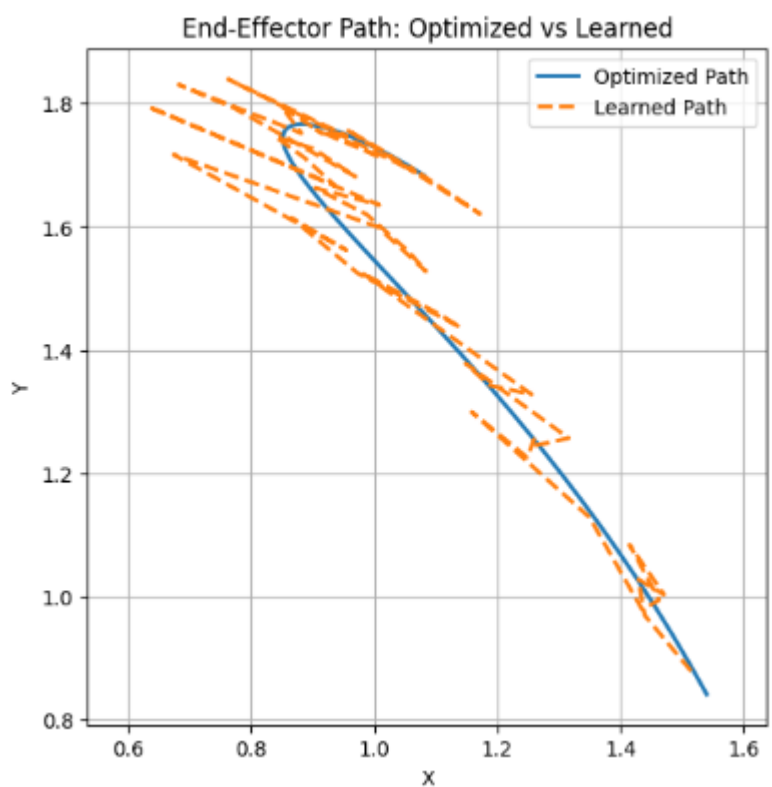
```
ax2.plot(x_pred, y_pred, '--', label="Predicted Path")
```

```
ax2.set_aspect("equal")
```

```
ax2.legend()
```

```
st.pyplot(fig2)
```





In this assignment, optimized joint-space trajectories generated using numerical optimization were used to train a learning-based model for a 2-link planar robot. A multilayer perceptron was trained to map start and end joint configurations to complete joint trajectories. The model was evaluated using mean squared error and showed good accuracy on test data. An interactive Streamlit dashboard was developed to compare optimized and predicted trajectories. Joint angle profiles and end-effector paths demonstrate that the learned trajectories closely approximate optimized ones. The learned model produces smooth motion while significantly reducing computation time. However, prediction accuracy decreases for configurations outside the training distribution. Learning-based trajectory generation is preferable when real-time performance is required. Optimization remains more reliable for high-precision or safety-critical motions.